

interesting items worth considering as a potential option for future use.

Black

Our main linter is Black. We use the default configuration of it.

There is a manual CI job in the review stage (Python section) that will lint the entire repo and return a non-zero exit code if files need to be formatted. It is up to both the MR author and the reviewer to make sure that this job passes before the MR is merged. To lint the entire repo, run the command:

```
bash
jump analytics
black .
```

If you want to check all files (without formatting) in the entire repo or any particular folder of file using black, run:

```
bash
for the entire repo, run
$ jump analytics
$ run black --check .
```

```
for particular folder, run
$ run black --check extract/
```

```
for particular folder, run
$ run black --check extract/saasusageping/usageping.py
```

mypy

- mypy

> Mypy is an optional static type checker for Python that aims to combine the benefits of dynamic (or duck) typing and static typing. Mypy combines the expressive power and convenience of Python with a powerful type system and compile-time type checking. Mypy type checks standard Python programs.

```
bash
Run mypy for extract/ folder
mypy extract/ --ignore-missing-imports
```

flake8

Your tool for style guide enforcement:

Flake8 is a popular lint wrapper for python. Under the hood, it runs three other tools and combines their results: pep8 for checking style. pyflakes for checking syntax. mccabe for

checking complexity.

```
bash
flake8 . --ignore=E203,E501,W503,W605
```

pylint

Pylint is a static code analyser for Python.

> Pylint analyses your code without actually running it. It checks for errors, enforces a coding standard, looks for code smells, and can make suggestions about how the code could be refactored. Pylint can infer actual values from your code using its internal code representation (astroid). If your code is `import logging as argparse`, Pylint will know that `argparse.error(...)` is in fact a logging call and not an argparse call.

Pylint is highly configurable and permits to write plugins in order to add your own checks (for example, for internal libraries or an internal rule). Pylint also has an ecosystem of existing plugins for popular frameworks and third party libraries.

```
bash
we use pylint and exclude some of check to reduce the noise (ie. line-too-long)
pylint extract/ --ignore=analytics/dags --disable=line-too-
long,E0401,E0611,W1203,W1202,C0103,R0801,R0902,W0212
```

xenon

Xenon is a monitoring tool based on Radon. It monitors your code's complexity. Ideally, Xenon is run every time you commit code. Through command line options, you can set various thresholds for the complexity of your code. It will fail (i.e. it will exit with a non-zero exit code) when any of these requirements is not met.

```
bash
run xenon --max-absolute B --max-modules B --max-average A . -i transform,sharedmodules
```

vulture

Vulture finds unused code in Python programs. This is useful for cleaning up and finding errors in large code bases. If you run Vulture on both your library and test suite you can find untested code.

Due to Python's dynamic nature, static code analyzers like Vulture are likely to miss some dead code. Also, code that is only called implicitly may be reported as unused. Nonetheless, Vulture can be a very helpful tool for higher code quality.

```
bash
run vulture . --min-confidence 100
```

Few more handy libraries

For more elements on how we automated linter to keep code quality on the elevated level, refer to · Python CI jobs. All of these linters can be tested automatically and for that purpose, we created a comprehensive set of commands using Makefile.

In addition, recommendation to check, explore and considering:

- pycodestyle
- yapf
- autopep8
- HowDOI - good tool for a quick search.
- iSort - isort is a Python utility / library to sort imports alphabetically, and automatically separated into sections and by type.

Tools for automating our coding quality standards

For automating code quality and testing, we are using our own product GitLab CI/CD pipeline. Details of pipelines we use for python should be found on the page CI jobs (Python).

When not to use Python

Since this style guide is for the entire data team, it is important to remember that there is a time and place for using Python and it is usually outside of the data modeling phase. Stick to SQL for data manipulation tasks where possible.

SQLAlchemy Upgrade - Codebase Changes

As of 2025-02-04, the analytics/ repo has been updated to use the latest dataimage, as detailed in Analytics MR!11537. This update includes upgrading several Python libraries, most notably sqlalchemy. The specific version installed is snowflake-sqlalchemy==1.6.1, which relies on sqlalchemy==2.0.

Changes in SQLAlchemy Query Patterns

The new version of SQLAlchemy enforces stricter rules on how queries can be passed. Below are examples of how Python statements should be updated to adhere to the updated SQLAlchemy library:

1. execute:

- Old: `connection.execute(query)`
- New: `gitlabdata.executequerystr(connection, query)`

2. readsql:

- Old: `pd.readsql(query)`
- New: `pd.readsql(text(query))`

3. hastable:

- Old: `engine.hastable(table)`

- New: `gitlabdata.hastable(engine, table)`

4. Creating a new engine:

- The `autocommit` parameter needs to be set explicitly when creating a new engine. The correct setting depends on the database being used.
- The `gitlabdata` library provides preset engines that can be used for convenience, i.e `snowflakeenginefactory` and `postgresenginefactory`

SECTION: enterprise-data/platform/snowflake-warehouse-optimization.md

Quick Summary

Guidelines to choose the correct warehouse for a particular dbt model:

- The user should base the initial estimated warehouse size on the number of partitions scanned within the query
- Benchmark by increasing the warehouse size and comparing the 'cost' vs 'reduction in query time'
- In the context of the total credits consumed by the dbt DAG, right-sizing warehouses by model for the smallest models does not lead to much cost reduction

Prospectus

In our current dbt set-up, we use a mix of Large and X-Large warehouses to run our production dbt jobs, based on the model tag.

However, dbt offers the ability to specify the warehouse size on the model level.

Initially, we had looked at right-sizing the warehouse for every single model, and we ran benchmarks in this issue to see what the potential cost savings would be. We found that right-sizing the warehouse for our smaller models doesn't make a key difference in the credits consumed when looking at overall credit consumption.

However, this does not make right-sizing the warehouse for models an unnecessary task. It just shows that we should be selective about which models we try to right-size. Since it takes time to right-size each model, time should be allocated to the models that are currently costing the most credits.

The rest of this page will document:

1. Key theoretical concepts for warehouse efficiency:
 - How warehouse sizing affects cost vs performance
 - What makes the most 'efficient' warehouse for our needs
 - Hypothetical examples
1. How to right-size a warehouse
 - Steps needed
 - Real-world examples
1. When to spend time right-sizing models

1. Future steps

Section 1: Warehouse sizing - concepts

<details><summary>Warehouse sizing - concepts</summary>

Concept 1: Cost vs Performance

Each time you increase the size of the warehouse by one level, you are doubling the CPU and RAM of your compute instance. Correspondingly, you are also doubling the price via the credits you consume.

While the price is doubled, in an ideal world on the proper workload/query, the query time is reduced by half due to the doubling of the compute instance's resources. Therefore, in this situation, you would pay the same price on either warehouse, but the bigger warehouse would halve the run-time of the query, making it the more efficient choice.

Eventually, if you keep upsizing the warehouse, you'll reach a point of diminishing returns, where the query run-time may be only slightly reduced while the costs increase disproportionately faster.

Here's a select.dev cost vs performance diagram illustrating this point [1].

Concept 2: Defining Warehouse efficiency

Warehouse Efficiency comes in two dimensions: cost and performance. Note that 'warehouse efficiency' is a different concept than model efficiency.

Because there are two dimensions, 'warehouse efficiency' doesn't have a clear definition; it needs to be decided based on how much you value 'cost' versus 'performance.'

In our case, we want one of these two outcomes when using a different warehouse:

- maintain existing runtimes while reducing costs
- reduce runtimes, without increasing cost drastically

Based on the requirements above, we will define warehouse efficiency clearly as follows:

> A warehouse is efficient when compared to the next-size smaller warehouse; the query run-time is reduced by 40% or more.

Explanation: As discussed in the previous section, for the next-size bigger warehouse to break even, it needs to run 2x as fast (query run-time is reduced by 50%) to compensate for its 2x price increase.

In our definition, we require run-time to be reduced by at least 40%, which translates to a maximum increase in cost of 20%. The reason why we don't require a 50% reduction in run-time (and break-even in costs) is the following:

- we need our performance to stay close to existing dbt run-times
- we want to work iteratively, which means setting a more realistic goal that allows some

wiggle-room and then adjusting the goalpost once we learn more.

<details><summary>Aside: calculating percentage improvements for run-times</summary>

To get into a technicality here, percentage improvement in query run-time can be a bit confusing because performance improves when run-time decreases - it's an inverse relationship.

We will calculate the percentage improvement in run-time using the traditional formula [2]:

```
$$
\frac{\text{new} - \text{old}}{\text{old}} \times 100\%
$$
```

For example, if we have the values new = 5 and old = 10, the calculation would be:

```
$$
\begin{align}
&\frac{\text{new} - \text{old}}{\text{old}} \times 100\% \quad \&= \quad \frac{5 - 10}{10} \times 100\% \quad \backslash\backslash \\
&\&= \frac{-5}{10} \times 100\% \quad \backslash\backslash \\
&\&= -50\%
\end{align}
$$
```

That means the new value is 50% smaller (faster, since we're talking about response time) than the old value.

</details>

That might sound a bit vague, so here are some hypothetical examples:

Hypothetical Examples

Example 1

The run-time has decreased by 50%, from 1 hour -> 30 minutes. In this case, it's a no-brainer to increase the warehouse size:

warehouse size	Credits/hr	run-time in hours	cost
X-small	1	1	\$2
Small	2	0.5	\$2.00
% increase		-50%	0%

Example 2

The run-time has decreased by 40% from 1 hour -> 36 minutes. Meanwhile, the cost has increased by 20% from \$2 to \$2.40:

warehouse size	Credits/hr	run-time in hours	cost

X-small	1	1	\$2.00	
Small	2	0.6	\$2.40	
% increase		-40%	20%	

In this case, upsizing the warehouse is no longer a no-brainer. It now depends on how much you value cost vs. query run-time.

If you only cared about cost, you would stick with the X-small; if you only cared about speed, you would go with the Small.

In our case, we want to find a balance between both, so we use the previously established guidelines:

> A warehouse is efficient when compared to the next-size smaller warehouse, the query run-time is 40% less.

In this case, since the query run-time has been reduced by 40%, it's exactly on the threshold of being considered 'efficient', and we should choose the 'S' warehouse.

Example 3

Example 3, run-time has decreased by 20% from 1 hour -> 48 minutes. Meanwhile, the cost has increased by 60%, from \$2 to \$3.20:

warehousesize	Credits/hr	run-time in hours	cost	
-----	-----	-----	-----	
X-small	1	1	\$2.00	
Small	2	0.8	\$3.20	
% increase		-20%	60%	

In this case, the query would be considered 'inefficient' because the runtime has only been reduced by 20%. For it to be considered 'efficient', the runtime needs to be reduced by 40% or more. Therefore, the 'XS' warehouse should be chosen.

</details>

Section 2: Steps to right-sizing warehouse

<details><summary>Steps to rightsizing warehouse</summary>

We have discussed in theory what an 'optimal' warehouse is, but how do we actually choose the correct warehouse for each model?

Here are the steps:

1. run an explain plan to figure out how many partitions the table has
1. Based on the number of partitions, use select.dev of partitions diagram to estimate what warehouse to start with
1. Run the query in dbt with the estimated warehouse from the previous step
1. Check the below 'Benchmarking heuristics' section to see if you can immediately assign a

warehouse; if not, continue

1. Using this Google Sheet record the query time

1. Upsizing warehouse(s)

- Now, rerun the query with the next highest warehouse
- Record the results in the Google Sheet again
- Stop once the cost is increasing faster than the performance

In the following sections, we'll look at the following:

- Why the benchmarking needs to happen in dbt
- We'll look at two examples of right-sizing warehouses by following these steps:
 1. simple query: gitlabdotcomdeploymenttododedupesource
 1. more complicated query with more partitions: prepcistage

<details><summary>Why benchmarking should be done in dbt environment</summary>

Benchmarking needs to be done in dbt

In the above list of steps, it states that the queries need to be run in dbt. This is because for almost all models, at least two sets of SQL statements are being run. Firstly, a SELECT statement is being run, but additionally, one of these two statements is also being run under the hood:

- For incremental models, a MERGE statement is being run
- For new and fullrefresh models, a CREATE TABLE statement is being run

These additional SQL statements can be expensive computationally and need to be part of the benchmark.

When using dbt run, these statements are automatically run for you. Therefore, it's best to run all warehouse benchmarking queries via dbt, either locally or via CI job.

Benchmarking incremental runs

When benchmarking incremental models specifically, the 'most efficient' warehouse will differ based on whether it's an incremental or fullrefresh run.

At the very least, the incremental runs should be benchmarked like so:

1. The corresponding table should be fully loaded

1. Delete the previous day of data and then run the model incrementally using the estimated warehouse based on the partition size.

Optional: The model could also be benchmarked by running it on fullrefresh as well. That way, the model can be configured on a different warehouse based on whether it is an incremental or fullrefresh run:

```
sql
{% if isincremental() %}
  {{ config(
```



```

        warehouse='smallerwarehouse' Use this warehouse for incremental runs
    ) }}
{% else %}
    {{ config(
        warehouse='largerwarehouse' Use this warehouse for full refresh runs
    ) }}
{% endif %}

```

```

SELECT
FROM mytable

```

</details>

<details><summary>Real world benchmarking examples</summary>

The below sections will look at two examples of how to follow the benchmarking steps.

example 1: gitlabdotcomdeploymenttododedupesource

The first step is to run an explain plan, which shows the query has 630 partitions:

```

sql
EXPLAIN
SELECT

FROM
"RAW".tappostgres.GITLABDBTODOS
QUALIFY ROWNUMBER() OVER ( PARTITION BY id ORDER BY uploadedat DESC) = 1;

```

The 2nd step is to choose a warehouse based on the number of partitions, per the above diagram. In this case, with 630 partitions, we can start with a size XS warehouse.

With warehouse XS, the query 01b5ff9d-080a-e214-0000-289d77d4f1e2 took 7m14s, so we'll add those stats to the Google Sheets template

Next, we will try using M warehouse. First, let's remove caching by running:

```

sql
ALTER SESSION SET USECACHEDRESULT = FALSE;

```

With warehouse M query 01b5ffc3-080a-e214-0000-289d77d57b3e took 1m45s, so we'll add those stats to the Google Sheets template:

warehouse size	Credits/hr	run-time minutes part	run-time seconds part	run-time in hours	cost
-----	-----	-----	-----	-----	-----

X-small	1	7	14	0.12
\$0.24				
Medium	4	1	45	0.03
\$0.23				
% increase				-76%
-3%				

The above shows that with the M warehouse, run-time has decreased by 73%, AND cost has also decreased by 3%, meaning that a 'M' warehouse is superior to the XS warehouse in this situation.

Since performance has increased faster than cost, we can upsize again to the L warehouse to see if that continues.

Using the L warehouse, the query 01b5ffdc-080a-e214-0000-289d77d5c756 finished in 1m23s, pasting this into the spreadsheet:

warehousesize	Credits/hr	run-time minutes part	run-time seconds part	run-time in hours	cost
-----	-----	-----	-----	-----	-----
X-small	1	7	14	0.12	
\$0.24					
Medium	4	1	45	0.03	
\$0.23					
Large	8	1	23	0.02	
\$0.37					
% increase					-21%
58%					

The above spreadsheet shows that query run-time was reduced by 21% (good), but costs increased by 58% (bad). Therefore, the price is growing much faster than the performance gains.

Conclusion: The M warehouse has the best run-time-to-cost ratio and should be used for this model.

example 2: prepcistage

<details><summary>explain plan</summary>

```
sql
EXPLAIN
WITH
  dimproject AS (
    SELECT

    FROM
      "PROD".common.dimproject
  ),
  dimpipeline AS (
```

```

SELECT

FROM
  "PROD".common.dimcipeline
),
dimnamespaceplanhist AS (
  SELECT

FROM
  "PROD".common.dimnamespaceplanhist
),
dimdate AS (
  SELECT

FROM
  "PROD".common.dimdate
),
cistages AS (
  SELECT

FROM
  "PREP".gitlabdotcom.gitlabdotcomcistagesdedupesource
WHERE
  createdat IS NOT NULL
),
joined AS (
  SELECT
    cistages.id AS dimcistageid,
    IFNULL(dimproject.dimprojectid, -1) AS dimprojectid,
    IFNULL(dimcipeline.dimcipelineid, -1) AS dimcipelineid,
    IFNULL(dimnamespaceplanhist.dimplanid, 34) AS implanid,
    IFNULL(dimnamespaceplanhist.dimnamespaceid, -1) AS ultimateparentnamespaceid,
    dimdate.dateid AS createddateid,
    cistages.createdat::TIMESTAMP AS createdat,
    cistages.updatedat::TIMESTAMP AS updatedat,
    cistages.name AS cistagename,
    cistages.status AS cistagestatus,
    cistages.lockversion AS lockversion,
    cistages.position AS POSITION
FROM
  cistages
  LEFT JOIN dimproject ON cistages.projectid = dimproject.dimprojectid
  LEFT JOIN dimnamespaceplanhist ON dimproject.ultimateparentnamespaceid =
dimnamespaceplanhist.dimnamespaceid
  AND cistages.createdat >= dimnamespaceplanhist.validfrom
  AND cistages.createdat < COALESCE(dimnamespaceplanhist.validto, '2099-01-01')
  LEFT JOIN dimcipeline ON cistages.pipelineid = dimcipeline.dimcipelineid
  INNER JOIN dimdate ON TODATE(cistages.createdat) = dimdate.dateday
)
SELECT

```

```
,
 '@mpeychet'::VARCHAR AS createdby,
 '@chrissharp'::VARCHAR AS updatedby,
 '2021-06-29'::DATE AS modelcreateddate,
 '2022-06-01'::DATE AS modelupdateddate,
 CURRENTTIMESTAMP() AS dbtupdatedat,
 CURRENTTIMESTAMP() AS dbtcreatedat
FROM
joined;
```

</details>

The EXPLAIN plan shows that there are a total of 9211 partitions that will need to be scanned. The estimation warehouse diagram has guidelines for 8,000 and 16,000 multi-partitions.

We'll start with the L warehouse.

Let's first run the below statement to prevent caching for benchmarking:

```
sql
ALTER SESSION SET USECACHEDRESULT = FALSE;
```

These were the query run-times:

- L: 32m17s
- XL: 11m32s

Pasting the run-times in our spreadsheet:

warehousesize	Credits/hr	run-time minutes part	run-time seconds part	run-time in hours	cost
Large	8	32	17	0.54	
\$17.22					
XL	16	11	32	0.19	
\$12.30					
% increase				-64%	
-29%					

It's clear that the XL is more 'efficient' here because the run-time was reduced by 64%, and the cost was reduced by 29%.

</details>

</details>

Section 3: When to right-size models

<details><summary>When to right-size models</summary>

As mentioned in the Prospectus section, a benchmark was done right-sizing all the models in the non-product dbt task, and per this benchmarking issue. It shows that right-sizing the smallest models does NOT lead to significant cost-savings (if at all), even though these type of models make up more than 90% of the models within the run.

Therefore, in the context of the dbt.py DAG, benchmarking would be most valuable for the most expensive models. We can use the queryattribution Snowflake table to find these models.

A small caveat to the above is that we also run dbt models in other environments outside of the production dbt.py DAG, such as:

- locally
- CI jobs

In these latter environments, it may make sense to right-size the warehouse for every model, even if the models are small. In these cases, the method to right-size doesn't need to be as precise, i.e., having to compare against multiple warehouse sizes. Instead, one could immediately assign an estimated warehouse per model based on the number of partitions scanned by the model.

To summarize, the priority for right-sizing warehouses should be the largest models that take the longest to run and consume the most credits since these models make up a disproportionate amount of the total credits consumed.

</details>

Future steps

1. When working on query optimization of the most expensive models, one should also consider benchmarking the warehouse to make sure the most 'efficient' warehouse is being used
1. Consider implementing these benchmarking steps programmatically. For example, a CI job could be one end product of this that tells the user what size warehouse they should be using.

Sources

- 1] [select.dev: warehouse sizing
- 2] calculating percentage improvements for run-times, [StackOverflow

SECTION: enterprise-data/platform/sql-style-guide.md

SQL Style Guide

This guide establishes our standards for SQL and are enforced by the SQLFluff linter and by code review. The target code changes that this stile guide apply to are those made using dbt.

If you are not on the Data Team or you are developing SQL outside of dbt, keep in mind that the

linting tools may be more difficult to apply, but you are welcome to follow the guidance given within this guide.

Usage

We expect that people will use the style presented in this guide during their development. Enforcement is still expected to happen at the time of review as the CI pipeline is only run manually. At a later date the pipeline will run with every change and eventually set to be mandatory for the pipeline to pass.

The previous style guide used a mixed indentation style that the linter does not support so every model will likely fail the first time it is checked. The intention is that models will be updated to the new style as they are worked on normally as not all of the rules can automatically be enforced (such as explicit column aliasing). The intention of working on updating them in the course of other development is to keep the changes manageable.

SQLFluff

SQLFluff is a SQL linter that works with templating tools like dbt. We use it to define the basic structure and style of the SQL that we write and move the review of that structure and style into the hands of the authors. SQLFluff is included within the dbt development environment and it uses the dbt templating engine during the linting process. It can be used with the following command:

```
console
sqlfluff lint models/path/to/file/file-to-lint.sql
```

If you are in the python virtual environment for dbt then a make command with a dbt model name can be used:

```
console
make lint-models MODEL="modelname"
```

A dbt command can also be used to get a list of files to lint:

```
console
sqlfluff lint $(dbt list --model modelname --output path)
```

If you are writing SQL that is not templated using dbt then you can install and use SQLFluff directly as it is a standalone python package.

```
console
pip install sqlfluff
sqlfluff lint path/to/file/file-to-lint.sql
```

SQLFluff includes a fix command that will apply fixes to rule violations when possible. Not all

rule violations are automatically fixable; therefore, you are encouraged to run the lint command after using the fix command to ensure that all rule violations have been resolved.

- SQLFluff Documentation
- SQLFluff Default configuration

Changes from the default configuration

- selecting the dialect to snowflake
- selecting the templater to be dbt
- excluding the following rules:
 - L016 Line length
 - L031 No table aliases in FROM or JOIN clauses
 - L034 Order of simple targets and calculations in SELECT statement
- aligning column aliases within the SELECT statement
- excluding the use of hanging indents
- selecting the tab size to be 2 spaces
- selecting Key words, Data Types and Functions to always be upper case
- require table aliases to be a minimum of four characters

The configuration file that the Data Team uses can be found in the GitLab Data Team repository.

General Guidance

- Do not optimize for fewer lines of code, new lines are cheap but brain time is expensive.
- Familiarize yourself with the DRY Principal. Leverage CTEs, jinja and macros in dbt, and snippets in Sisense. If you type the same line twice, it needs to be maintained in two places.
- Be consistent. Even if you are not sure of the best way to do something do it the same way throughout your code, it will be easier to read and make changes if they are needed.
- Be explicit. Defining something explicitly will ensure that it works the way you expect and it is easier for the next person, which may be you, when you are explicit in SQL.

Best Practices

- No tabs should be used - only spaces. Your editor should be setup to convert tabs to spaces - see our onboarding template for more details.
- Wrap long lines of code, between 80 and 100, to a new line.
- Do not use the USING command in joins because it produces inaccurate results in Snowflake. Create an account to view the forum discussion on this topic.
- Understand the difference between the following related statements and use appropriately:
 - UNION ALL and UNION
 - LIKE and ILIKE
 - NOT and ! and <>
 - DATEPART() and DATETRUNC()

- Use the AS operator when aliasing a column or table.
- Prefer DATEDIFF to inline additions datecolumn + intervalcolumn. The function is more explicit and will work for a wider variety of date parts.
- Prefer != to <>. This is because != is more common in other programming languages and reads like "not equal" which is how we're more likely to speak.
- Prefer LOWER(column) LIKE '%match%' to column ILIKE '%Match%'. This lowers the chance of stray capital letters leading to an unexpected result.
- Prefer WHERE to HAVING when either would suffice.

Commenting

- When making single line comments in a model use the -- syntax
- When making multi-line comments in a model use the / / syntax
- Respect the character line limit when making comments. Move to a new line or to the model documentation if the comment is too long
- Utilize the dbt model documentation when it is available
- Calculations made in SQL should have a brief description of what's going on and if available, a link to the handbook defining the metric (and how it's calculated)
- Instead of leaving TODO comments, create new issues for improvement

Naming Conventions

- An ambiguous field name such as id, name, or type should always be prefixed by what it is identifying or naming:

```
sql
-- Preferred
SELECT
  id AS accountid,
  name AS accountname,
  type AS accounttype,
  ...
```

-- vs

```
-- Not Preferred
SELECT
  id,
  name,
  type,
  ...
```

- All field names should be snake-cased:


```

sql
-- Preferred
SELECT
    dvcecreatedtstamp AS devicecreatedtimestamp
    ...

```

-- VS

```

-- Not Preferred
SELECT
    dvcecreatedtstamp AS DeviceCreatedTimestamp
    ...

```

- Boolean field names should start with has, is, or does:

```

sql
-- Preferred
SELECT
    deleted AS isdeleted,
    sla    AS hassla
    ...

```

-- VS

```

-- Not Preferred
SELECT
    deleted,
    sla,
    ...

```

- Timestamps should end with at and should always be in UTC.
- Dates should end with date.
- When truncating dates name the column in accordance with the truncation.

```

sql

SELECT
    originalat,                -- 2020-01-15 12:15:00.00
    originaldate,             -- 2020-01-15
    DATETRUNC('month',originaldate) AS originalmonth -- 2020-01-01
    ...

```

- Avoid key words like date or month as a column name.

Reference Conventions

- When joining tables and referencing columns from both tables consider the following:
 - reference the full table name instead of an alias when the table name is shorter, maybe less than 20 characters. (try to rename the CTE if possible, and lastly consider aliasing to something descriptive)
 - always qualify each column in the SELECT statement with the table name / alias for easy navigation

```
sql
-- Preferred
SELECT
    budgetforecast.accountid,
    datedetails.fiscalyear,
    datedetails.fiscalquarter,
    datedetails.fiscalquartername,
    costcategory.costcategorylevel1,
    costcategory.costcategorylevel2
FROM budgetforecastcogsopex AS budgetforecast
LEFT JOIN datedetails
    ON datedetails.firstdayofmonth = budgetforecast.accountingperiod
LEFT JOIN costcategory
    ON budgetforecast.uniqueaccountname = costcategory.uniqueaccountname
```

-- vs

```
-- Not Preferred
SELECT
    a.accountid,
    b.fiscalyear,
    b.fiscalquarter,
    b.fiscalquartername,
    c.costcategorylevel1,
    c.costcategorylevel2
FROM budgetforecastcogsopex a
LEFT JOIN datedetails b
    ON b.firstdayofmonth = a.accountingperiod
LEFT JOIN costcategory c
    ON b.uniqueaccountname = c.uniqueaccountname
```

- Only use double quotes when necessary, such as columns that contain special characters or are case sensitive.

```
sql
-- Preferred
SELECT
```

```

    "FirstName&" AS firstname,
    ...

-- VS

-- Not Preferred
SELECT
    FIRSTNAME AS firstname,
    ...

```

- Prefer accessing JSON using the bracket syntax.

```

sql
-- Preferred
SELECT
    databyrow['id']::bigint as idvalue
    ...

-- VS

-- Not Preferred
SELECT
    databyrow:"id"::bigint as idvalue
    ...

```

- Prefer explicit join statements.

```

sql
-- Preferred
SELECT
FROM firsttable
INNER JOIN secondtable
...

-- VS

-- Not Preferred
SELECT
FROM firsttable,
    secondtable
...

```

- Also prefer explicit join statements for LATERAL FLATTEN. However, it should be noted that the current code-base does not consistently adhere to this practice.

```

sql

```

```

-- Preferred
SELECT
    data.value,
    source.uploadedat
FROM source
INNER JOIN LATERAL FLATTEN(input => source.jsontext['data']) AS data
...

-- vs

-- Not Preferred
SELECT
    data.value,
    source.uploadedat
FROM source,
    LATERAL FLATTEN(input => source.jsontext['data']) AS data
...

```

Common Table Expressions (CTEs)

- Prefer CTEs over sub-queries as CTEs make SQL more readable and are more performant:

```

sql
-- Preferred
WITH importantlist AS (

    SELECT DISTINCT
        specificcolumn
    FROM othertable
    WHERE specificcolumn != 'foo'

)

SELECT
    primarytable.column1,
    primarytable.column2
FROM primarytable
INNER JOIN importantlist
    ON primarytable.column3 = importantlist.specificcolumn

-- vs

-- Not Preferred
SELECT
    primarytable.column1,
    primarytable.column2
FROM primarytable
WHERE primarytable.column3 IN (
    SELECT DISTINCT specificcolumn

```

```
FROM othertable
WHERE specificcolumn != 'foo')
```

- Use CTEs to reference other tables.
- CTEs should be placed at the top of the query.
- Where performance permits, CTEs should perform a single, logical unit of work.
- CTE names should be as concise as possible while still being clear.
 - Avoid long names like `replacesfdcaccountidwithmasterrecordid` and prefer a shorter name with a comment in the CTE. This will help avoid table aliasing in joins.
- CTEs with confusing or notable logic should be commented in file and documented in dbt docs.
- CTEs that are duplicated across models should be pulled out into their own models.

Data Types

- Use default data types and not aliases. Review the Snowflake summary of data types for more details. The defaults are:
 - NUMBER instead of DECIMAL, NUMERIC, INTEGER, BIGINT, etc.
 - FLOAT instead of DOUBLE, REAL, etc.
 - VARCHAR instead of STRING, TEXT, etc.
 - TIMESTAMP instead of DATETIME

The exception to this is for timestamps. Prefer TIMESTAMP to TIME. Note that the default for TIMESTAMP is TIMESTAMPNTZ which does not include a time zone.

Functions

- Prefer IFNULL to NVL.
- Prefer IFF to a single line CASE statement:

```
sql
-- Preferred
SELECT
  IFF(column1 = 'foo', column2,column3) AS logicswitch,
  ...
```

-- vs

```
-- Not Preferred
SELECT
  CASE
    WHEN column1 = 'foo' THEN column2
    ELSE column3
  END AS logicswitch,
  ...
```

- Prefer IFF to selecting a boolean statement:

```

sql
-- Preferred
SELECT
    IFF(amount < 10,TRUE,FALSE) AS islessthanten,
    ...
-- vs

```

```

-- Not Preferred
SELECT
    (amount < 10) AS islessthanten,
    ...

```

- Prefer simplifying repetitive CASE statements where possible:

```

sql
-- Preferred
SELECT
    CASE fieldid
        WHEN 1 THEN 'date'
        WHEN 2 THEN 'integer'
        WHEN 3 THEN 'currency'
        WHEN 4 THEN 'boolean'
        WHEN 5 THEN 'variant'
        WHEN 6 THEN 'text'
    END AS fieldtype,
    ...

```

```

-- vs

```

```

-- Not Preferred
SELECT
    CASE
        WHEN fieldid = 1 THEN 'date'
        WHEN fieldid = 2 THEN 'integer'
        WHEN fieldid = 3 THEN 'currency'
        WHEN fieldid = 4 THEN 'boolean'
        WHEN fieldid = 5 THEN 'variant'
        WHEN fieldid = 6 THEN 'text'
    END AS fieldtype,
    ...

```

- Prefer the explicit date function over datepart, but prefer datepart over extract:

```

sql
DAYOFWEEK(createdat) > DATEPART(dayofweek, 'createdat') > EXTRACT(dow FROM createdat)

```

- Be mindful of date part interval when using the DATEDIFF function as the function will only return whole interval results.

Example Code

This example code has been processed though SQLFluff linter and had the style guide applied.

sql

WITH mydata AS (

```
SELECT mydata.  
FROM prod.mydatawithalongtablename AS mydata  
INNER JOIN prod.otherthing  
WHERE mydata.filter = 'myfilter'
```

),

somecte AS (

```
SELECT DISTINCT  
    id AS otherid,  
    otherfield1,  
    otherfield2,  
    datefieldat,  
    databyrow,  
    field4,  
    field5,  
    LAG(  
        otherfield2  
    ) OVER (PARTITION BY otherid, otherfield1 ORDER BY 5) AS previousotherfield2  
FROM prod.myotherdata
```

),

/

This is a very long comment: It is good practice to leave comments in code to explain complex logic in CTEs or business logic which may not be intuitive to someone who does not have intimate knowledge of the data source. This can help new users familiarize themselves with the code quickly.

/

final AS (

```
SELECT  
    -- This is a singel line comment  
    mydata.field1 AS detailedfield1,  
    mydata.field2 AS detailedfield2,  
    mydata.detailedfield3,  
    DATETRUNC('month', somecte.datefieldat) AS datefieldmonth,  
    somecte.databyrow['id']::NUMBER AS idfield,
```

```

IFF(mydata.detailedfield3 > mydata.field2, TRUE, FALSE) AS isboolean,
CASE
  WHEN mydata.cancellationdate IS NULL
    AND mydata.expirationdate IS NOT NULL
    THEN mydata.expirationdate
  WHEN mydata.cancellationdate IS NULL
    THEN mydata.startdate + 7 -- There is a reason for this number
  ELSE mydata.cancellationdate
END AS adjustedcancellationdate,
COUNT() AS numberofrecords,
SUM(somecte.field4) AS field4sum,
MAX(somecte.field5) AS field5max
FROM mydata
LEFT JOIN somecte
  ON mydata.id = somecte.id
WHERE mydata.field1 = 'abc'
  AND (mydata.field2 = 'def' OR mydata.field2 = 'ghi')
GROUP BY 1, 2, 3, 4, 5, 6
HAVING COUNT() > 1
ORDER BY 8 DESC
)

SELECT
FROM final

```

Other SQL Style Guides

- Brooklyn Data Co
- dbt Labs
- Matt Mazur
- Kickstarter

SECTION: enterprise-data/platform/ci-for-ds-pipelines/index.md

Our Approach to Using CI/CD For Data Science

When it comes to training, scoring, and deploying machine learning models, there are trade-offs to using your local machine vs. CI. Our approach is flexible to allow users to do both.

- When executing training or scoring runs on your local machine, users will be able to:
 - Execute training or scoring code using local resources and python builds specific to the machine
 - Log, view, and retrieve experiments in your preferred experiment tracker
 - Upload and download model artifacts to your preferred model/package registry

- Commit changes to the repository that will not trigger a CI pipeline
- When executing training or scoring runs remotely using GitLab CI/CD, users will be able to:
 - Select CPU or GPU runners based on the size and needs of the model
 - Automatically detect and rebuild the model image based on changes to Dockerfile or requirements.txt
 - Use different containers for different models in development, staging, and production
 - Log, view, and retrieve experiments in your preferred experiment tracker
 - Upload and download model artifacts directly to the GitLab model registry
 - Automatically report model metrics and performance in the merge request for review by others.
- Commit changes to the repository and automatically trigger CI pipelines based on a specific commit message
 - Training CI pipelines only execute with the following commit message: train <path/to/notebook/yournotebook.ipynb>
 - Scoring CI pipeline only executes with the following commit message: score <path/to/notebook/yournotebook.ipynb>
 - Allows the pipeline to execute just the desired notebook
 - Allow training and scoring CI pipelines to run at set dates and times using Scheduled pipelines
 - Log results to Project wiki
 - Use GitLab for Slack integration to monitor pipeline status

Advantages of Using CI for Training Data Science Models

- Reproducibility
- Automation
- Speed
- Separate development, staging, and production environments
- Logging results directly to Merge Request and Project Wiki
- Scalable GPU and CPU resources
- Scheduling
- Slack notifications for monitoring CI pipelines

Getting Started

This section covers, in detail, the mechanisms behind how these pipelines is created and configured. If you are just interested in getting your data science CI training pipeline up and running, skip directly to the Model Training Step-by-Step Instructions

If you are just interested in getting your data science CI scoring pipeline up and running, skip directly to the Scoring and Productionalization Step-by-Step Instructions

Key Repository Files

Within our public GitLab Data Science CI Example repository are the following key files:

- .gitlab-ci.yml: This is the CI/CD configuration file that defines the jobs that will be run for each pipeline. The actual pipelines are pulled from the CI/CD Component Catalog, with only the variables that need specified are set in this .yml.

- Dockerfile: Instructions for creating the docker image used in the Container Registry. Here we are using python 3.9 running on Ubuntu 22.04 with CUDA drivers for GPU
- requirements.txt: The python packages to install in the container
- trainingconfig.yaml: Configuration for training notebook
- scoringconfig.yaml: Configuration for scoring notebook
- notebooks/trainingexample.ipynb: training notebook used for this example
- notebooks/scoringexample.ipynb: scoring code notebook used for this example
- xgbmodel.json: The saved model from training that will be used for scoring (Note: in future iterations this will be pulled directly from the Model Registry)

Model Training with CI/CD

Training Pipeline

1. Build

- build-new-image: activated whenever changes are made to the Dockerfile or requirements.txt files. This will rebuild the image used in the MR to train the model
- build-mr-image: Checks if a image exists in the Container Registry for the MR. If not, one will be built.

2. Train

- train-commit-activated: To execute a training pipeline. Activated by using the train <path/to/notebook/yournotebook.ipynb> commit message

3. Notify (optional)

- publish-metrics-comment: Write model metrics as a comment to the merge request. This is executed after a training or scoring run is performed through the commit message method.

4. Test (optional)

- secret-detection: Pipeline secret detection to scan for potential exposed secrets in your pipeline

Training Setup

Let's take a detailed look at the repository (Code -> Repository):

- In the notebooks directory, open trainingexample.ipynb. We do not need to change anything in here, but note the first cell of the notebook. Papermill will be used to execute this notebook in the CI pipeline and this cell has been tagged as parameters to allow Papermill to change these values. There are 3 variables with preset values:
 - treemethod = 'auto': The default value for xgboost. The CI pipeline changes this value to gpuhist when using a GPU runner in CI to take advantage certain performance enhancements. The assumption that is the model will train with CPU when run locally due to lack of a compatible local GPU, or when on a CPU runner
 - notebookdir = 'notebooks': The directory of the training notebooks. This value is the same locally as it is the CI pipeline.
- To see exactly how the CI pipeline changes these the treemethod variable, let's view .gitlab-ci.yml. Because we are using a component from the CI/CD catalog, we actually want to look at the Data Science ML Component Pipeline. In this file, search for train-commit-activated. This is the most important stage in the CI pipeline. A few things to note:
 - image: \$CONTAINERIMAGE:\$IMAGETAG: The training job will use the container created in the

build job (as defined in build-mr-image), using the Dockerfile and requirements.txt files in the repository.

- tags: Determines which runner to use. We want to be able to change the runner based on the project needs. This gets specified by TRAINRUNNER in .gitlab-ci.yml located back in our project repository

- script:

...

- papermill ... -p treemethod 'gpuhist' \$notebookName ...: Tells Papermill to override the variable values defined in the first cell of the specified notebook with the values shown when utilizing a the GPU runner.

- Finally, let's look at the trainingconfig.yaml. Here we can configure certain variables for training our model:

- outcome: Our outcome/target/dv variable. The example notebook is using the breast cancer dataset from scikit-learn and the outcome field in that dataset is named target

- optuna configurations: The example notebook runs an xgboost model with Optuna. There are a lot of customizations possible with this setup, but to keep it simple we have only included:

- ntrials: Number of trails to run in the Optuna study

- modelfilename: The output name of the model file

- mlflow: There are a few configurations you can make here if you like, but the defaults will also work fine for this example

- experimentname: Name of your MLFlow Experiment

- runname: ID or name of the MLFlow Experiment Run

Step-by-Step Instructions

1. Fork the public GitLab Data Science CI Example repository. Forking will allow you to further customize the code to meet your own needs.

1. Optional (but recommended) Configurations:

- Experiment Tracker: This will allow you to log your experiments in the native Experiment Tracker (or MLFlow instance) and log the model artifacts to the package registry.

- Review the MLflow Client Compatibility Instructions to set up the MLFLOWTRACKINGURI and MLFLOWTRACKINGTOKEN CI/CI in your project.

- Create a project access token (Settings -> Access Tokens -> Add New Token) named REPOTOKEN with a Developer role and the following scopes: api, readapi, readrepository, writerepository. Be sure to copy this token.

- Note: Enabling group access tokens is not available for SaaS Free accounts. If using a Free account, you will need to fork the project into a personal (instead of a group) namespace

- Create the following new CI Variables (Settings -> CI/CD -> Variables -> Add New Variable):

- MLFLOWTRACKINGTOKEN: For the value, enter the project access token value created above.

- MLFLOWTRACKINGURI: For the value, use the GitLab API MLFlow endpoint as outlined in the MLFlow instructions above. It should look something like:

<https://gitlab.com/api/v4/projects/<yourprojectid>/ml/mlflow>. Project ID can be found in Settings -> General

- Note: For all variables, deselect the "Protect Variable" flag to enable experiment tracking on unprotected branches. Tick "Mask variable" to prevent the value from showing in the logs.

- Write Model Metrics to Merge Request

- Create the following new CI/CD Variable (Settings -> CI/CD -> Variables -> Add New Variable):

- REPOTOKEN: For the value, enter the project access token value created above.
- Note: De-select the "Protect Variable" flag to enable experiment tracking on unprotected branches. Tick "Mask variable" to prevent the value from showing in the logs.

1. Now, let's make some changes to activate our training pipeline:

1. Create a new branch (Code -> Branches -> New Branch)

- width="500" alt=""/>

1. (Optional) Configure your runners:

- GPU runners are available for Premium and Ultimate users. If enabled, edit .gitlab-ci.yml and change the value of TRAINRUNNER to a GPU runner (e.g. saas-linux-medium-amd64-gpu-standard).

- The default value saas-linux-small-amd64 will work for all account types.

1. Edit trainingconfig.yaml

- Let's make a change so that we have something to commit. Change ntrials to a new value between 10 and 20.

- For the commit message enter train notebooks/trainingexample.ipynb. This will tell the GitLab that you want to execute the training CI pipeline on the trainingexample.ipynb notebook found in the notebooks directory. Commit the change.

1. Click "Create merge request". Make sure you are merging into your local fork and click "Create merge request" once again. This should activate the training CI pipeline for the newly created MR.

1. Click on "Pipelines" and you should see the training pipeline running. Click into the pipeline to see which stage the pipeline is in.

- Note: If you did not set up the step above "Write Model Metrics to Merge Request", then the publish-metrics-comment job will fail. The pipeline will still pass with warnings

1. Once the pipeline has finished, you will see a new comment posted on the merge request that contains some model metrics from the run (assuming you set up Write Model Metrics to Merge Request).

- width="700" alt=""/>

1. Now let's look at the experiment run we just completed with our CI pipeline (Analyze -> Model Experiments)

- Click on your experiment name.

- You should see a new run logged from the CI Pipeline. Click into that run.

- Run details are displayed, including a link to the CI job, the merge request, various parameters and metrics, and model artifacts.

- Click on "Artifacts". This will take you to the Package Registry, where all the artifacts associated with that particular model run are stored. You should see the .json model file, a .yaml configuration file, and a requirements.txt. These can be used later to deploy your model.

1. Finally, let's look at the container that was used to train the model (Deploy -> Container Registry)

- By default, the container datascience-images is used (this can be change this in .gitlab-ci.yml through the DSCONTAINERIMAGE optional input)

- Click into datascience-images and you should see a new tag that was used to score the model in the merge requests. This tag was created by the CI pipeline, and each MR and model release will have it's own container tag.

Model Scoring with CI/CD

Scoring Pipeline

1. Build

- build-new-image: activated whenever changes are made to the Dockerfile or requirements.txt files. This will rebuild the image used in the MR to score the model
- build-mr-image: Checks if a image exists in the Container Registry for the MR. If not, one will be built.

2. Score

- score-commit-activated: To manually execute a scoring pipeline. Activated by using score <path/to/notebook/yournotebook.ipynb> in the commit message
- score-scheduled: To execute a scoring pipeline based on a defined schedule using Scheduled pipelines

3. Notify (Optional)

- publish-metrics-comment: Write model performance metrics as a comment on the merge request. This is executed after a scoring run is performed through the commit message method.
- write-to-wiki: Write model performance metrics and job details to the project wiki. The CI configuration it set up to only execute for scheduled jobs that use the score-scheduled job

Scoring Setup

Let's take a detailed look at the repository (Code -> Repository):

- In the notebooks directory, open scoringexample.ipynb. We do not need to change anything in here, but note the first cell of the notebook. Papermill will be used to execute this notebook in the CI pipeline and this cell has been tagged as parameters to allow Papermill to change these values. There are 3 variables with preset values:
- To see exactly how the CI pipeline changes these values, now let's view .gitlab-ci.yml. Because we are using a component from the CI/CD catalog
- notebookdir = 'notebooks': The directory of the scoring notebook (can be the same directory as the training notebooks). This value is the same locally as it is the CI pipeline.
- To see exactly how the CI pipeline changes these values, now let's view .gitlab-ci.yml. Because we are using a component from the CI/CD catalog, we actually want to look at the Data Science ML Component Pipeline. In this file, search for score-commit-activated. A few things to note:
- This job will manually trigger when score <path/to/notebook/yournotebook.ipynb> is passed in the commit message.
- image: \$CONTAINERIMAGE:\$IMAGETAG: The training job will use the container created in the build job (as defined in build-mr-image), using the Dockerfile and requirements.txt files in the repository.
- tags: Determines which runner to use. We want to be able to change the runner based on the project needs. This gets specified by SCORERUNNER in .gitlab-ci.yml located back in our project repository
- There is also a score-scheduled job, which we cover in more detail in Model Deployment
- This will trigger the scoring notebook at a set time, using Scheduled pipelines
- This job will also trigger the write-to-wiki job, which will publish model metrics to the project wiki
- Finally, let's look at the scoringconfig.yaml. Here we can configure certain variables for training our model:

- **modelfile**: The model file created during training that will be used to score the model. This could also be pulled directly from the Model Registry, but for simplicity, we are including it directly in the repository
- **fields**: List of the model fields. This is useful if the model is using only a subset of fields in a dataframe. In this example, all the fields in the dataframe are being used.
- **decilecuts**: Sometimes it's useful to include decile cuts from the validation dataset used during training. In this example, we included quintile cuts to illustrate how they can be used.

Step-by-Step Instructions

1. Optional (but recommended) Wiki Configurations:

- This allows you to log your scheduled runs to the GitLab Project Wiki.
- Create a new CI/CD Variable (Settings -> CI/CD -> Variables -> Add New Variable):
 - **APIENDPOINT**: For the value, use the GitLab API endpoint (will be similar to the **MLFLOWTRACKINGURI** set up during training) using the following format: <https://gitlab.com/api/v4/projects/<yourprojectid>>. Project ID can be found in Settings -> General.

- Note: De-select the "Protect Variable" flag to enable experiment tracking on unprotected branches

1. Configure your runner:

- GPU runners are available for Premium and Ultimate users. If enabled, edit `.gitlab-ci.yml` and change the value of **SCORERUNNER** to a GPU runner (e.g. `saas-linux-medium-amd64-gpu-standard`).
- The default value `saas-linux-small-amd64` will work for all account types.

1. Now lets score our model using CI/CD

- You can either create a new branch of the repository or use the same one as used above for training the model.
- Make a change to `notebooks/scoringexample.ipynb` so that we have something to commit. This could be as simple as adding a line to one of the cells.
- For the commit message enter `score notebooks/scoringexample.ipynb`. This will tell the GitLab that you want to execute the `score-commit-activated` CI pipeline on the `scoringexample.ipynb` notebook found in the `notebooks` directory. Commit the change.
- Click "Create merge request". Make sure you are merging into your local fork and click "Create merge request" once again. This should activate the scoring CI pipeline for the newly created MR.
- Click on "Pipelines" and you should see the scoring pipeline running. Click into the pipeline to see which stage the pipeline is in.
- Note: If you did not set up the step above "Write Model Metrics to Merge Request", then the `publish-metrics-comment` job will fail. The pipeline will still pass with warnings
- Once the pipeline has finished, you will see a new comment posted on the merge request that contains some model metrics from the run (assuming you set up `Write Model Metrics to Merge Request`). This is the same process you would have seen during training, except it is now writing out descriptives about the scored dataset.

Model Deployment With CI/CD

Now that we have our model trained and scoring pipeline set up, we can focus on deploying our model. Here we are going to focus on batch scoring so that the model scores are updated in batch once a day.

Advantages of Model Deployment in GitLab

- Centralized Model Management: GitLab's Experiment Tracker and Model Registry provides a centralized location for storing, versioning, and managing machine learning models.
- Automated Deployment: Use GitLab CI/CD to automate the deployment process, ensuring consistent and repeatable deployments across environments.
- Scalability: GitLab's infrastructure allows for easy scaling of your model serving capabilities as demand grows. As the model grows, the size of the runner can be increased.
- Reproducibility: By using GitLab's version control and CI/CD pipelines, you can ensure that your model training, scoring, and deployment processes are reproducible.
- Documentation and Logging: Monitor model pipelines, receive slack notifications when failures occur, and log key metrics directly to the project wiki.
- Security: Leverage GitLab's security features to ensure your models and data are protected throughout the deployment process.
- Continuous Improvement: Models can be fine-tuned, retrained, and tested without disrupting models in production.
- Cost-Effective: Resources are only consumed when the model runs, reducing the cost of running your models.

Step-by-Step Instructions

1. Create a new tagged release

- Creating a tag will allow us to capture the state of the model at a specific point in time. Doing so will build a version of the model that will be deployed to production that won't be affected by changes to the repository after that point in time.

- Navigate to Code -> Tag -> New Tag
- Create a new tag (something like 1.0.0) and click Create Tag
- This will trigger the deployment pipeline, which is build a new container tagged with a production- prefix (e.g. production-1.0.0)

1. Setup a pipeline schedule

- Navigate to Build -> Pipeline schedules -> New schedule
- Give your pipeline a description
- Set when you want it to run. In the example below, the pipeline is scheduled to run every day at noon.
- Set target tag to 1.0.0. This is the tag we just created.
- We need to setup one variable, SCORINGNOTEBOOK with the location of the notebook we want to schedule notebooks/scoringexample.ipynb
- Tick "Activated" and save the changes.
- 

1. Test out your schedule by manually triggering it

- Should see the pipeline start to run
- It will use the production container than was created when you created the 1.0.0 tag, named production-1.0.0

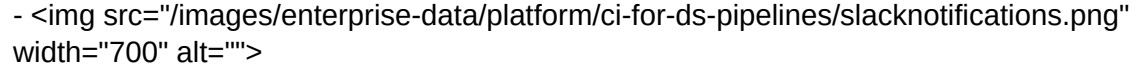
1. View the logs and model metrics

- Navigate to Plan -> Wiki and you will see a list by timestamp of all the times the scheduled pipeline has run, with links to the job logs and model metrics.
- 

1. Configure Slack notifications (optional):

- In your GitLab project, go to Settings -> Integrations -> GitLab for Slack
- Follow the instructions in the GitLab For Slack app documentation
- We've setup our slack notifications so that notifications are sent to our data-science-pipelines channel only when a pipeline fails. If a pipeline succeeds, a notification is not sent.
- width="700" alt="Diagram showing the setup for Slack notifications in GitLab CI/CD. It illustrates the flow from a pipeline failure to a Slack notification in the 'data-science-pipelines' channel." data-bbox="114 158 842 188"/>
- Save your changes

Slack Notifications (optional)

- In Slack, add the GitLab for Slack app
- Follow the instructions in the GitLab For Slack app documentation
- We've setup our slack notifications so that notifications are sent to our data-science-pipelines channel only when a pipeline fails. If a pipeline succeeds, a notification is not sent.
- width="700" alt="Diagram showing the setup for Slack notifications in GitLab CI/CD. It illustrates the flow from a pipeline failure to a Slack notification in the 'data-science-pipelines' channel." data-bbox="114 341 830 371"/>

And that's it! Feel free to modify these pipelines and notebooks to fit your data science modeling needs. And be sure to check out all the other great data science resources on our Data Science Handbook Page. If you are experiencing any difficulty or if you have any suggestions to improve these pipelines, feel free to open an issue with us. Happy pipelining!

```
---
## SECTION: enterprise-data/platform/ci-jobs/index.md
---
```

This page documents the CI jobs used by the data team in Merge Requests in both the Data Tests and Analytics projects.

What to do if a pipeline fails

- If a weekend has passed re-run any CLONE steps which were performed prior, every Sunday (5:00AMUTC) all old pipeline databases are dropped from Snowflake older than 14 days.
- Merge master branch. Due to how dbt handles packages pipelines can fail due to package failures which should always be handled in the latest branch.
- Confirm model selection syntax. In general, it is easiest to simply use the file names of the models you are changing.
- If still uncertain or facing any issues, request assistance in the data Slack channel

Variable Name not found in the CI Pipeline job

This kind of error pops up in the pipeline like `KeyError: 'GITLABCOMCIDBUSER'`. It means the variable is not defined in the variable section of CI/CD Settings. To resolve this, add the variable name to CI/CD setting i.e. settings --> cicd --> variable, also provide the variable value.

Notes:- Turn off the Flags, so the variable is accessible from the CI pipeline.
The same applies to the variable value; if it is incorrect in the job, we can update it in the above link.

Analytics pipelines

Stages

CI jobs are grouped by stages.

.. Snowflake

These jobs are defined in .gitlab-ci.yml. All Snowflake objects created by a CI clone job will exist until dropped, either manually or by the weekly clean up of Snowflake objects.

cloneprepspecificschema

Run this if you need a clone of any schema available in the prep database. Specify which schema to clone with the SCHEMANAME variable. If the clone already exists, this will do nothing.

cloneprodspecificschema

Run this if you need a clone of any schema available in the prod database. Specify which schema to clone with the SCHEMANAME variable. If the clone already exists, this will do nothing.

cloneprod

Runs automatically when the MR opens to be able to run any dbt jobs. Subsequent runs of this job will be fast as it only verifies if the clone exists. This is an empty clone of the prod and prep databases.

cloneprodreal

Run this if you need to do a real clone of the prod and prep databases. This is a full clone both databases.

clonerawfull

Run this if you need to run extract, freshness, or snapshot jobs. Subsequent runs of this job will be fast as it only verifies if the clone exists.

clonerawpostgrespipeline

Run this if you only need a clone of the raw tappostgres schema in order to test changes to postgres pipeline or a manifest file. If the raw clone already exists, this will do nothing.

clonerawsheetload

Run this if you only need a clone of the raw sheetload schema in order to test changes or additions to sheetload. If the raw clone already exists, this will do nothing.

clonerawspecificschema

Run this if you need a clone of any other raw schema in order to test changes or additions. Specify which raw schema to clone with the SCHEMANAME variable. If the raw clone already exists, this will do nothing.

clonerawbyschema

Clones the entire RAW DB, created due to timeout issues when trying to clone the DB using SF commands.

NB Due to the size of the DB created by running, only run this when you absolutely have to run through complete platform tests. Likely only applicable for infrastructure upgrades.

forcecloneboth

Run this if you want to force refresh raw, prod, and prep. This does a full clone of raw, but a shallow clone of prep and prod.

· Extract

These jobs are defined in extract-ci.yml

boneyardsheetload

Run this if you want to test a new boneyard sheetload load. This requires the real prod and prep clones to be available.

sheetload

Run this if you want to test a new sheetload load. This jobs runs against the clone of RAW. Requires the clonerawspecificschema (parameter SCHEMANAME=SHEETLOAD) job to have been run.

· gitlabsaaspptest

This pipeline needs to be executed when doing changes to any of the below manifest files present in path analytics/extract/gitlabsaaspostgrespipeline/manifests:

1. elsaascustomersscddbmanifest.yaml
1. elgitlabdotcomdb